Nama : Khania Puji Auliya

NIM : 254107020236

Kelas : 1G

Prodi : D-IV Teknik Informatika

LAPORAN JOBSHEET 10

PERCOBAAN 1

- **Kode Program MahasiswaL15**

```
public class MahasiswaL15 {
    String nim;
    String nama;
    String kelas;
    double ipk;

    public MahasiswaL15() {
    }
    public MahasiswaL15(String nm, String name, String kls, double ip) {
        this.nim = nm;
        this.nama = name;
        this.kelas = kls;
        this.ipk = ip;
    }
    void tampilInformasi() {
        System.out.println(nama + "\t\t" + nim + "\t" + kelas + "\t" + ipk);
    }
}
```

- **Kode Program Node15**

```
public class Node15 {
    MahasiswaL15 data;
    Node15 next;

    public Node15 (MahasiswaL15 data, Node15 next) {
        this.data = data;
        this.next = next;
    }
}
```

- **Kode Program SingleLinkedList15**

```
public class SingleLinkedList15 {
    Node15 head;
    Node15 tail;
```

```
boolean isEmpty() {
    return (head == null);
}
public void print() {
    if (!isEmpty()) {
        Node15 tmp = head;
        System.out.println("Isi Linked List:\t");
        while (tmp != null) {
            tmp.data.tampilInformasi();
            tmp = tmp.next;
        }
        System.out.println("");
    } else {
        System.out.println("Linked list kosong");
    }
}
public void addFirst(MahasiswaL15 input) {
    Node15 ndInput = new Node15 (input, null);
    if (isEmpty()) {
        head = ndInput;
        tail = ndInput;
    } else {
        ndInput.next = head;
        head = ndInput;
    }
}
public void addLast(MahasiswaL15 input) {
    Node15 ndInput = new Node15(input, null);
    if (isEmpty()) {
        head = ndInput;
        tail = ndInput;
    } else {
        tail.next = ndInput;
        tail = ndInput;
    }
}
public void insertAfter(String key, MahasiswaL15 input) {
    Node15 ndInput = new Node15(input, null);
    Node15 temp = head;
    do {
        if (temp.data.nama.equalsIgnoreCase(key)) {
            ndInput.next = temp.next;
            temp.next = ndInput;
```

```
            if (ndInput.next == null) {
                tail = ndInput;
            }
            break;
        }
        temp = temp.next;
    } while (temp != null);
}
public void insertAt(int index, MahasiswaL15 input) {
    if (index < 0) {
        System.out.println("indeks salah");
    } else if (index == 0) {
        addFirst(input);
    } else {
        Node15 temp = head;
        for (int i = 0; i < index - 1; i++) {
            temp = temp.next;
        }
        temp.next = new Node15(input, temp.next);
        if (temp.next.next == null) {
            tail = temp.next;
        }
    }
}
}
```

- **Kode Program SLLMain15**

```
public class SLLMain15 {
    public static void main(String[] args) {
        SingleLinkedList15 sll = new SingleLinkedList15();

        MahasiswaL15 mhs1 = new MahasiswaL15("24212200", "Alvaro", "1A", 4.0);
        MahasiswaL15 mhs2 = new MahasiswaL15("23212201", "Bimon", "2B", 3.8);
        MahasiswaL15 mhs3 = new MahasiswaL15("22212202", "Cintia", "3C", 3.5);
        MahasiswaL15 mhs4 = new MahasiswaL15("21212203", "Dirga", "4D", 3.6);

        sll.print();
        sll.addFirst(mhs4);
        sll.print();
        sll.addLast(mhs1);
        sll.print();
        sll.insertAfter("Dirga", mhs3);
        sll.insertAt(2, mhs2);
        sll.print();
```

```
    }
}
```

- **Hasil Running**

```
Linked list kosong
Isi Linked List:
Dirga       21212203     4D    3.6

Isi Linked List:
Dirga       21212203     4D    3.6
Alvaro      24212200     1A    4.0

Isi Linked List:
Dirga       21212203     4D    3.6
Cintia      22212202     3C    3.5
Bimon       23212201     2B    3.8
Alvaro      24212200     1A    4.0
```

Pertanyaan

1. Mengapa hasil compile kode program di baris pertama menghasilkan "Linked List Kosong"?
2. Jelaskan kegunaan variable temp secara umum pada setiap method!
3. Lakukan modifikasi agar data dapat ditambahkan dari keyboard!

Jawaban

1. Karena dalam kode program terdapat head = null, yang artinya belum ada data di dalam linked list.
2. Variabel temp digunakan sebagai pointer sementara untuk menelusuri setiap node tanpa mengubah head, sehingga memudahkan proses pencarian, penambahan, penghapusan, dan penampilan data.
3. Modifikasi

   - **Kode Program**

```
import java.util.Scanner;
public class SLLMain15 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        SingleLinkedList15 sll = new SingleLinkedList15();

        System.out.print("Jumlah data mahasiswa: ");
        int jumlah = sc.nextInt();
        sc.nextLine();
```

```
        for (int i = 0; i < jumlah; i++) {
            System.out.println("\nData Mahasiswa ke-" + (i + 1));
            System.out.print("NIM   : ");
            String nim = sc.nextLine();
            System.out.print("Nama  : ");
            String nama = sc.nextLine();
            System.out.print("Kelas : ");
            String kelas = sc.nextLine();
            System.out.print("IPK   : ");
            double ipk = sc.nextDouble();
            sc.nextLine();
            MahasiswaL15 mhs = new MahasiswaL15(nim, nama, kelas, ipk);
            sll.addLast(mhs);
        }
        System.out.println("\nData Linked List:");
        sll.print();
    }
}
```

- **Hasil Running**

```
Jumlah data mahasiswa: 4

Data Mahasiswa ke-1
NIM   : 24214400
Nama  : Alvaro
Kelas : 1A
IPK   : 4,0

Data Mahasiswa ke-2
NIM   : 23213301
Nama  : Bimon
Kelas : 2B
IPK   : 3,8

Data Mahasiswa ke-3
NIM   : 22212202
Nama  : Cintia
Kelas : 3C
IPK   : 3,5

Data Mahasiswa ke-4
NIM   : 21212203
Nama  : Dirga
```

```
Kelas : 4D
IPK   : 3,6

Data Linked List:
Isi Linked List:
Alvaro      24214400     1A    4.0
Bimon       23213301     2B    3.8
Cintia      22212202     3C    3.5
Dirga       21212203     4D    3.6
```

PERCOBAAN 2

- **Kode Program SingeLinkedList15**

```
public class SingleLinkedList15 {
    Node15 head;
    Node15 tail;

    boolean isEmpty() {
        return (head == null);
    }
    public void print() {
        if (!isEmpty()) {
            Node15 tmp = head;
            System.out.println("Isi Linked List:\t");
            while (tmp != null) {
                tmp.data.tampilInformasi();
                tmp = tmp.next;
            }
            System.out.println("");
        } else {
            System.out.println("Linked list kosong");
        }
    }
    public void addFirst(MahasiswaL15 input) {
        Node15 ndInput = new Node15 (input, null);
        if (isEmpty()) {
            head = ndInput;
            tail = ndInput;
        } else {
            ndInput.next = head;
            head = ndInput;
        }
    }
    public void addLast(MahasiswaL15 input) {
```

```
    Node15 ndInput = new Node15(input, null);
    if (isEmpty()) {
        head = ndInput;
        tail = ndInput;
    } else {
        tail.next = ndInput;
        tail = ndInput;
    }
}
public void insertAfter(String key, MahasiswaL15 input) {
    Node15 ndInput = new Node15(input, null);
    Node15 temp = head;
    do {
        if (temp.data.nama.equalsIgnoreCase(key)) {
            ndInput.next = temp.next;
            temp.next = ndInput;
            if (ndInput.next == null) {
                tail = ndInput;
            }
            break;
        }
        temp = temp.next;
    } while (temp != null);
}
public void insertAt(int index, MahasiswaL15 input) {
    if (index < 0) {
        System.out.println("indeks salah");
    } else if (index == 0) {
        addFirst(input);
    } else {
        Node15 temp = head;
        for (int i = 0; i < index - 1; i++) {
            temp = temp.next;
        }
        temp.next = new Node15(input, temp.next);
        if (temp.next.next == null) {
            tail = temp.next;
        }
    }
}
public void getData(int index) {
    Node15 tmp = head;
    for (int i = 0; i < index; i++) {
        tmp = tmp.next;
```

```
    }
    tmp.data.tampilInformasi();
}
public int indexOf(String key) {
    Node15 tmp = head;
    int index = 0;
    while (tmp != null && !tmp.data.nama.equalsIgnoreCase(key)) {
        tmp = tmp.next;
        index++;
    }
    if (tmp == null) {
        return -1;
    } else {
        return index;
    }
}
public void removeFirst() {
    if (isEmpty()) {
        System.out.println("Linked List masih kososng, tidak dapat dihapus!");
    } else if (head == tail) {
        head = tail = null;
    } else {
        head = head.next;
    }
}
public void removeLast() {
    if (isEmpty()) {
        System.out.println("Linked List masih kososng, tidak dapat dihapus!");
    } else if (head == tail) {
        head = tail = null;
    } else {
        Node15 temp = head;
        while (temp.next != tail) {
            temp = temp.next;
        }
        temp.next = null;
        tail = temp;
    }
}
public void remove(String key) {
    if (isEmpty()) {
        System.out.println("Linked List masih kososng, tidak dapat dihapus!");
    } else {
        Node15 temp = head;
```

```
            while (temp != null) {
                if ((temp.data.nama.equalsIgnoreCase(key)) && (temp == head)) {
                    this.removeFirst();
                    break;
                } else if (temp.data.nama.equalsIgnoreCase(key)) {
                    temp.next = temp.next.next;
                    if (temp.next == null) {
                        tail = temp;
                    }
                    break;
                }
                temp = temp.next;
            }
        }
    }
    public void removeAt(int index) {
        if (index == 0) {
            removeFirst();
        } else {
            Node15 temp = head;
            for (int i = 0; i < index -1; i++) {
                temp = temp.next;
            }
            temp.next = temp.next.next;
            if (temp.next == null) {
                tail = temp;
            }
        }
    }
}
```

- **Kode Program SLLMain15**

```
import java.util.Scanner;
public class SLLMain15 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        SingleLinkedList15 sll = new SingleLinkedList15();

        System.out.print("Jumlah data mahasiswa: ");
        int jumlah = sc.nextInt();
        sc.nextLine();

        for (int i = 0; i < jumlah; i++) {
            System.out.println("\nData Mahasiswa ke-" + (i + 1));
```

```
            System.out.print("NIM   : ");
            String nim = sc.nextLine();
            System.out.print("Nama  : ");
            String nama = sc.nextLine();
            System.out.print("Kelas : ");
            String kelas = sc.nextLine();
            System.out.print("IPK   : ");
            double ipk = sc.nextDouble();
            sc.nextLine();
            MahasiswaL15 mhs = new MahasiswaL15(nim, nama, kelas, ipk);
            sll.addLast(mhs);
        }
        System.out.println("\nData Linked List:");
        sll.print();

        System.out.println("Data index 1: ");
        sll.getData(1);

        System.out.println("Data mahasiswa an Bimon berada pada index: " +
sll.indexOf("Bimon"));
        System.out.println();

        sll.removeFirst();
        sll.removeLast();
        sll.print();
        sll.removeAt(0);
        sll.print();
    }
}
```

- **Hasil Running**

```
Jumlah data mahasiswa: 4

Data Mahasiswa ke-1
NIM   : 21212203
Nama  : Dirga
Kelas : 4D
IPK   : 3,6

Data Mahasiswa ke-2
NIM   : 22212202
Nama  : Cintia
Kelas : 3C
IPK   : 3,5
```

```
Data Mahasiswa ke-3
NIM   : 23212201
Nama  : Bimon
Kelas : 2B
IPK   : 3,8

Data Mahasiswa ke-4
NIM   : 24212200
Nama  : Alvaro
Kelas : 1A
IPK   : 4,0

Data Linked List:
Isi Linked List:
Dirga      21212203      4D     3.6
Cintia     22212202      3C     3.5
Bimon      23212201      2B     3.8
Alvaro     24212200      1A     4.0

Data index 1:
Cintia     22212202      3C     3.5
Data mahasiswa an Bimon berada pada index: 2

Isi Linked List:
Cintia     22212202      3C     3.5
Bimon      23212201      2B     3.8

Isi Linked List:
Bimon      23212201      2B     3.8
```

Pertanyaan

1. Mengapa digunakan keyword break pada fungsi remove? Jelaskan!
2. Jelaskan kegunaan kode dibawah pada method remove

```java
temp.next = temp.next.next;
if (temp.next == null) {
    tail = temp;
}
```

Jawaban

1. Perintah break digunakan untuk menghentikan perulangan setelah data ditemukan agar proses lebih efisien dan tidak melakukan pengecekan lanjutan yang tidak diperlukan.
2. Kode temp.next = temp.next.next; digunakan untuk menghapus node setelah temp dengan cara melewatinya. Jika setelah temp.next berniali null, maka tail diperbarui menjadi temp karena menjadi node terakhir.

TUGAS

- **Kode Program MahasiswaQL15**

```
public class MahasiswaQL15 {
    String nim, nama, kelas;
    double ipk;

    public MahasiswaQL15(String nim, String nama, String kelas, double ipk) {
        this.nim = nim;
        this.nama = nama;
        this.kelas = kelas;
        this.ipk = ipk;
    }

    void tampilInformasi() {
        System.out.println(nama + "\t\t" + nim + "\t" + kelas + "\t" + ipk);
    }
}
```

- **Kode Program NodeQueue15**

```
public class NodeQueue15 {
    MahasiswaQL15 data;
    NodeQueue15 next;

    public NodeQueue15(MahasiswaQL15 data) {
        this.data = data;
        this.next = null;
    }
}
```

- **Kode Program QueueLinkedList15**

```
public class QueueLinkedList15 {
    NodeQueue15 front, rear;
    int size;
    int max = 100;
```

```
public QueueLinkedList15() {
    front = rear = null;
    size = 0;
}
boolean isEmpty() {
    return size == 0;
}
boolean isFull() {
    return size == max;
}
void clear() {
    front = rear = null;
    size = 0;
    System.out.println("Queue berhasil dikosongkan");
}
void enqueue(MahasiswaQL15 data) {
    if (isFull()) {
        System.out.println("Queue penuh");
        return;
    }
    NodeQueue15 newNode = new NodeQueue15(data);
    if (isEmpty()) {
        front = rear = newNode;
    } else {
        rear.next = newNode;
        rear = newNode;
    }
    size++;
    System.out.println("Data berhasil masuk queue");
}
void dequeue() {
    if (isEmpty()) {
        System.out.println("Queue kosong");
        return;
    }
    System.out.println("Data keluar:");
    front.data.tampilInformasi();
    front = front.next;
    size--;
    if (front == null) {
        rear = null;
    }
}
void peekFront() {
```

```
        if (isEmpty()) {
            System.out.println("Queue kosong");
        } else {
            System.out.println("Data terdepan:");
            front.data.tampilInformasi();
        }
    }
    void peekRear() {
        if (isEmpty()) {
            System.out.println("Queue kosong");
        } else {
            System.out.println("Data terakhir:");
            rear.data.tampilInformasi();
        }
    }
    void tampilSemua() {
        if (isEmpty()) {
            System.out.println("Queue kosong");
        } else {
            System.out.println("Isi Queue:");
            NodeQueue15 temp = front;
            while (temp != null) {
                temp.data.tampilInformasi();
                temp = temp.next;
            }
        }
    }
    void jumlahAntrian() {
        System.out.println("Jumlah antrian: " + size);
    }
}
```

- **Kode Program QueueLLMain15**

```
import java.util.Scanner;

public class QueueLLMain15 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        QueueLinkedList15 queue = new QueueLinkedList15();
        int pilih;

        do {
            System.out.println("\n=== MENU QUEUE ===");
            System.out.println("1. Menambah Antrian");
```

```
System.out.println("2. Memanggil Antrian");
System.out.println("3. Tampil Terdepan & Terakhir");
System.out.println("4. Tampil Semua");
System.out.println("5. Jumlah Antrian");
System.out.println("6. Kosongkan Antrian");
System.out.println("0. Keluar");
System.out.print("Pilih: ");
pilih = sc.nextInt();
sc.nextLine();

switch (pilih) {
    case 1:
        System.out.print("NIM   : ");
        String nim = sc.nextLine();
        System.out.print("Nama  : ");
        String nama = sc.nextLine();
        System.out.print("Kelas : ");
        String kelas = sc.nextLine();
        System.out.print("IPK   : ");
        double ipk = sc.nextDouble();
        sc.nextLine();

        MahasiswaQL15 mhs = new MahasiswaQL15(nim, nama, kelas, ipk);
        queue.enqueue(mhs);
        break;

    case 2:
        queue.dequeue();
        break;

    case 3:
        System.out.println("\n--- Data Terdepan ---");
        queue.peekFront();

        System.out.println("\n--- Data Terakhir ---");
        queue.peekRear();
        break;

    case 4:
        queue.tampilSemua();
        break;

    case 5:
        queue.jumlahAntrian();
```

```
                break;

            case 6:
                queue.clear();
                break;

            case 0:
                System.out.println("Terima kasih");
                break;

            default:
                System.out.println("Pilihan tidak valid");
        }
    } while (pilih != 0);
  }
}
```

- **Hasil Running**

```
=== MENU QUEUE ===
1. Menambah Antrian
2. Memanggil Antrian
3. Tampil Terdepan & Terakhir
4. Tampil Semua
5. Jumlah Antrian
6. Kosongkan Antrian
0. Keluar
Pilih: 1
NIM   : 24212200
Nama  : Alvaro
Kelas : 1A
IPK   : 4,0
Data berhasil masuk queue

=== MENU QUEUE ===
1. Menambah Antrian
2. Memanggil Antrian
3. Tampil Terdepan & Terakhir
4. Tampil Semua
5. Jumlah Antrian
6. Kosongkan Antrian
0. Keluar
Pilih: 1
NIM   : 23212201
Nama  : Bimon
```

```
Kelas : 2B
IPK   : 3,8
Data berhasil masuk queue

=== MENU QUEUE ===
1. Menambah Antrian
2. Memanggil Antrian
3. Tampil Terdepan & Terakhir
4. Tampil Semua
5. Jumlah Antrian
6. Kosongkan Antrian
0. Keluar
Pilih: 1
NIM   : 22212202
Nama  : Cintia
Kelas : 3C
IPK   : 3,5
Data berhasil masuk queue

=== MENU QUEUE ===
1. Menambah Antrian
2. Memanggil Antrian
3. Tampil Terdepan & Terakhir
4. Tampil Semua
5. Jumlah Antrian
6. Kosongkan Antrian
0. Keluar
Pilih: 1
NIM   : 21212203
Nama  : Dirga
Kelas : 4D
IPK   : 3,6
Data berhasil masuk queue

=== MENU QUEUE ===
1. Menambah Antrian
2. Memanggil Antrian
3. Tampil Terdepan & Terakhir
4. Tampil Semua
5. Jumlah Antrian
6. Kosongkan Antrian
0. Keluar
Pilih: 2
```

```
Data keluar:
Alvaro        24212200       1A     4.0

=== MENU QUEUE ===
1. Menambah Antrian
2. Memanggil Antrian
3. Tampil Terdepan & Terakhir
4. Tampil Semua
5. Jumlah Antrian
6. Kosongkan Antrian
0. Keluar
Pilih: 3

--- Data Terdepan ---
Data terdepan:
Bimon         23212201       2B     3.8

--- Data Terakhir ---
Data terakhir:
Dirga         21212203       4D     3.6

=== MENU QUEUE ===
1. Menambah Antrian
2. Memanggil Antrian
3. Tampil Terdepan & Terakhir
4. Tampil Semua
5. Jumlah Antrian
6. Kosongkan Antrian
0. Keluar
Pilih: 4
Isi Queue:
Bimon         23212201       2B     3.8
Cintia        22212202       3C     3.5
Dirga         21212203       4D     3.6

=== MENU QUEUE ===
1. Menambah Antrian
2. Memanggil Antrian
3. Tampil Terdepan & Terakhir
4. Tampil Semua
5. Jumlah Antrian
6. Kosongkan Antrian
0. Keluar
```

```
Pilih: 5
Jumlah antrian: 3

=== MENU QUEUE ===
1. Menambah Antrian
2. Memanggil Antrian
3. Tampil Terdepan & Terakhir
4. Tampil Semua
5. Jumlah Antrian
6. Kosongkan Antrian
0. Keluar
Pilih: 6
Queue berhasil dikosongkan

=== MENU QUEUE ===
1. Menambah Antrian
2. Memanggil Antrian
3. Tampil Terdepan & Terakhir
4. Tampil Semua
5. Jumlah Antrian
6. Kosongkan Antrian
0. Keluar
Pilih: 4
Queue kosong

=== MENU QUEUE ===
1. Menambah Antrian
2. Memanggil Antrian
3. Tampil Terdepan & Terakhir
4. Tampil Semua
5. Jumlah Antrian
6. Kosongkan Antrian
0. Keluar
Pilih: 0
Terima kasih
```